Instituto Federal de Brasília

Curso de Tecnologia em Análise e Desenvolvimento de Sistemas

Disciplina: Sistemas Computacionais

Trabalho Prático

Atividade em grupo: Observação de uma aplicação em execução

Tema

Ambiente de execução, processos, arquivos, rede, contêineres e monitoramento básico de uma aplicação.

Contexto

Em sistemas computacionais, uma aplicação não funciona de forma isolada. Ela depende do sistema operacional, de arquivos, processos, memória, processador, portas de rede, permissões, bibliotecas, serviços auxiliares e, em muitos casos, contêineres.

A atividade proposta foi inspirada em um projeto de monitoramento e observabilidade de aplicações em Kubernetes, no qual os estudantes devem escolher uma aplicação baseada em microsserviços, preparar um ambiente de execução, realizar testes de carga e observar métricas de desempenho. A proposta original trabalha com aplicações conteinerizadas, cluster Kubernetes, testes de carga, Prometheus, Grafana, escalabilidade e observabilidade.

Nesta disciplina, a atividade será adaptada para um nível introdutório, com foco em compreender como uma aplicação utiliza recursos do computador e como esses recursos podem ser observados.

Objetivos da atividade

Ao final da atividade, o grupo deverá ser capaz de:

- compreender como uma aplicação é executada pelo sistema operacional;
- identificar processos relacionados à aplicação;
- observar consumo de CPU e memória;
- identificar arquivos, diretórios e dependências do projeto;
- compreender o uso de portas de rede em servidores locais;

- diferenciar execução local, execução em contêiner e execução em ambiente mais estruturado;
- produzir documentação técnica simples sobre o experimento;
- refletir sobre desempenho, organização, limitações e boas práticas.

Organização dos grupos

A atividade deverá ser realizada em grupos de 3 a 4 estudantes.

Cada grupo deverá dividir as responsabilidades entre seus integrantes, garantindo que todos participem da execução, documentação e apresentação do experimento.

Sugestão de divisão de papéis:

- **Responsável pela aplicação**: organiza ou implementa a aplicação escolhida;
- **Responsável pelo ambiente**: prepara terminal, linguagem, dependências e execução;
- **Responsável por processos e recursos**: observa CPU, memória e processos;
- **Responsável por rede**: identifica portas, acesso local e requisições;
- **Responsável pela documentação**: consolida relatório, imagens, comandos e conclusões.

Observação

A divisão de papéis não significa que cada estudante fará apenas uma parte isolada. Todos devem compreender o experimento completo.

Aplicação a ser utilizada

Cada grupo deverá escolher ou desenvolver uma aplicação simples para ser executada localmente.

A aplicação pode ser:

- uma aplicação Flask simples;
- uma aplicação Node.js/Express;
- um servidor HTTP simples em Python;
- uma aplicação web estática servida localmente;
- uma aplicação em contêiner Docker;

- outra aplicação simples, desde que aprovada pelo professor.

A aplicação deve permitir algum tipo de acesso pelo navegador ou por requisição HTTP.

Exemplo didático

Uma aplicação Flask mínima pode responder em:

```
http://localhost:5000
```

Um servidor HTTP simples em Python pode responder em:

```
http://localhost:8000
```

Etapa 1: Preparação do ambiente

O grupo deverá preparar o ambiente necessário para executar a aplicação.
O relatório deve indicar:

- sistema operacional utilizado;
- linguagem de programação;
- versão da linguagem;
- editor de código utilizado;
- comandos executados no terminal;
- bibliotecas ou dependências instaladas;
- estrutura de arquivos e diretórios do projeto.

Exemplo didático

Comandos possíveis para uma aplicação Flask:

```
mkdir atividade-sistemas
cd atividade-sistemas
python3 -m venv venv
source venv/bin/activate
pip install flask
```

O grupo deverá registrar a estrutura do projeto, por exemplo:

```
atividade-sistemas/
 app.py
 requirements.txt
 README.md
 static/
 style.css
```

Etapa 2: Execução da aplicação

Após preparar o ambiente, o grupo deverá executar a aplicação.
O relatório deve responder:

1. Qual comando iniciou a aplicação?
2. Qual processo foi criado pelo sistema operacional?
3. Qual usuário executou esse processo?
4. A aplicação ficou em execução contínua ou terminou rapidamente?
5. Ela ficou aguardando requisições, entrada do usuário ou outro evento?

Exemplo didático

Para iniciar um servidor Flask:

```
python app.py
```

Para iniciar um servidor HTTP simples:

```
python3 -m http.server 8000
```

Etapa 3: Observação de processos

Com a aplicação em execução, o grupo deverá observar os processos do sistema.
Em Linux, podem ser usados comandos como:

```
ps aux
top
htop
pstree
```

Em Windows, podem ser usadas ferramentas ou comandos como:

```
tasklist
taskmgr
netstat -ano
```

O grupo deverá registrar:

- o nome do processo relacionado à aplicação;
- o PID do processo;
- o consumo aproximado de CPU;
- o consumo aproximado de memória;
- se há outros processos auxiliares relacionados;
- como o processo foi encerrado.

Observação

Nem todo processo aberto consome CPU continuamente. Muitos processos ficam aguardando entrada do usuário, leitura de arquivos, resposta de rede ou requisições externas.

Etapa 4: Observação de arquivos e diretórios

O grupo deverá analisar a organização dos arquivos da aplicação.
Devem ser identificados:

- arquivos de código-fonte;
- arquivos de configuração;
- arquivos de dependências;
- diretórios usados pela aplicação;
- arquivos temporários ou gerados durante a execução;
- arquivos que não deveriam ser enviados a um repositório público.

Questões para análise:

1. A estrutura de diretórios está organizada?
2. O projeto possui arquivo de dependências, como `requirements.txt` ou `package.json`?

3. Existem arquivos sensíveis, como `.env`, senhas ou tokens?
4. Que arquivos deveriam entrar no `.gitignore`?
5. O que aconteceria se um arquivo importante fosse movido para o diretório errado?

Etapa 5: Observação de rede

Se a aplicação usar servidor local, o grupo deverá identificar a porta de execução e testar o acesso.

Devem ser registrados:

- endereço local utilizado;
- porta usada pela aplicação;
- protocolo utilizado;
- forma de teste no navegador ou terminal;
- comportamento da aplicação quando recebe requisições.

Exemplo didático

Testes possíveis:

```
curl http://localhost:5000
curl http://127.0.0.1:8000
```

Também é possível verificar portas em uso.

Em Linux:

```
ss -tulnp
```

Em Windows:

```
netstat -ano
```

Etapa 6: Teste simples de carga

A proposta original prevê testes de carga com diferentes quantidades de usuários simultâneos e análise de métricas como throughput, latência, CPU, memória e taxa de erro.

Nesta atividade introdutória, o grupo deverá realizar um teste simples, sem necessidade de infraestrutura complexa.

O grupo pode usar uma das opções:

- várias requisições manuais pelo navegador;
- várias requisições com `curl`;
- ferramenta simples como `ab`, `k6` ou `Locust`, caso o grupo já tenha familiaridade;
- script simples em Python para fazer múltiplas requisições.

Exemplo com `curl`:

```
for i in {1..20}; do curl http://localhost:5000; done
```

O grupo deverá observar:

- a aplicação continuou respondendo?
- houve aumento no uso de CPU?
- houve aumento no uso de memória?
- houve erro em alguma requisição?
- a resposta ficou perceptivelmente mais lenta?

Etapa 7: Execução opcional com Docker

Caso o grupo já tenha contato com Docker, poderá executar a aplicação em um contêiner.

Nesse caso, deverá criar:

- um arquivo `Dockerfile`;
- uma imagem Docker;
- um contêiner em execução;
- mapeamento de portas;
- documentação dos comandos utilizados.

Exemplo de comandos:

```
docker build -t app-sistemas .
docker run --name app-sistemas -p 5000:5000 app-sistemas
docker ps
docker logs app-sistemas
docker stop app-sistemas
docker rm app-sistemas
```

O grupo deverá comparar:

1. O que mudou entre executar localmente e executar em contêiner?
2. O processo aparece da mesma forma no sistema operacional?
3. Como a porta foi mapeada?
4. O que aconteceria com arquivos criados dentro do contêiner?
5. Que vantagens e dificuldades o Docker trouxe para a atividade?

Etapa 8: Discussão sobre monitoramento e observabilidade

A proposta original solicita o uso de Prometheus e Grafana para observar métricas da aplicação em Kubernetes, incluindo requisições por segundo, latência, tempo de resposta, uso de CPU, uso de memória, quantidade de pods, consultas ao banco e erros HTTP ou gRPC.

Nesta adaptação, o grupo deverá discutir o conceito de monitoramento em nível introdutório.

O relatório deve responder:

1. Que informações foram observadas diretamente no sistema?
2. Quais métricas ajudaram a entender o comportamento da aplicação?
3. Quais informações não foram fáceis de observar?
4. Que tipo de ferramenta seria útil para monitorar melhor a aplicação?
5. Por que monitorar CPU, memória, rede, logs e erros é importante em sistemas reais?

Entregas

Cada grupo deverá entregar um arquivo compactado contendo:

- código-fonte da aplicação;
- arquivos de configuração utilizados;
- instruções de execução;
- relatório em PDF;
- prints ou registros dos comandos executados;
- vídeo curto de apresentação do experimento, se solicitado pelo professor.

A proposta original prevê entrega de códigos, instruções de uso, relatório e vídeo gravado pelos membros do grupo, com demonstração da participação e dos conhecimentos adquiridos.

Estrutura sugerida do relatório

O relatório deve conter:

1. identificação da instituição, disciplina, turma, data e integrantes do grupo;
2. introdução, apresentando o objetivo da atividade;
3. descrição da aplicação escolhida;
4. descrição do ambiente utilizado;
5. estrutura de arquivos e diretórios;
6. comandos utilizados para instalação e execução;
7. observação dos processos;
8. observação de CPU e memória;
9. observação de portas e rede;
10. teste simples de carga;
11. comparação entre execução local e execução com Docker, se realizada;
12. dificuldades encontradas;
13. soluções adotadas;
14. conclusão do grupo;
15. comentário individual de cada integrante sobre sua participação e aprendizado;
16. referências utilizadas.

A estrutura segue a lógica da atividade original, que solicita relatório com identificação, introdução, metodologia, seções sobre as fases do experimento, conclusão, comentários pessoais, referências e anexos quando necessário.

Critérios de avaliação

A atividade será avaliada considerando:

Critério	Peso
Funcionamento da aplicação escolhida	15%
Organização dos arquivos e clareza da estrutura do projeto	10%
Registro correto dos comandos utilizados	10%
Análise de processos, CPU e memória	15%
Análise de rede, portas e requisições	15%
Teste simples de carga e interpretação dos resultados	15%
Qualidade do relatório e documentação	10%
Participação equilibrada dos integrantes	10%

Observação

Na proposta original, a avaliação considera a qualidade das entregas e o nível técnico de exploração das solicitações feitas. Também são valorizadas boas descobertas, equilíbrio na distribuição das tarefas e boa documentação.

Questões para reflexão

Ao final da atividade, o grupo deverá responder:

1. O que acontece no sistema operacional quando a aplicação é executada?
2. Qual é a diferença entre o arquivo do programa e o processo em execução?
3. A aplicação consome CPU continuamente ou apenas quando recebe requisições?
4. Que arquivos são essenciais para a aplicação funcionar?
5. Como a estrutura de diretórios influencia a manutenção do projeto?
6. Qual porta foi utilizada pela aplicação? O que aconteceria se essa porta já estivesse em uso?
7. Que sinais indicam que uma aplicação está consumindo recursos excessivos?
8. Por que logs são importantes para diagnosticar problemas?
9. Se a aplicação fosse publicada para usuários reais, que cuidados adicionais seriam necessários?

10. Como essa atividade ajuda a relacionar sistema operacional, processos, arquivos, rede, terminal e desenvolvimento de software?

Produto final esperado

Ao final, o grupo deverá demonstrar que compreendeu não apenas como executar uma aplicação, mas também como observar seu comportamento dentro do sistema computacional.

A entrega deve evidenciar:

- domínio básico do terminal;
- compreensão de processos;
- organização de arquivos;
- noção de portas e requisições HTTP;
- interpretação inicial de uso de CPU e memória;
- documentação técnica clara;
- reflexão crítica sobre o funcionamento da aplicação.